Analysis of Algoritms

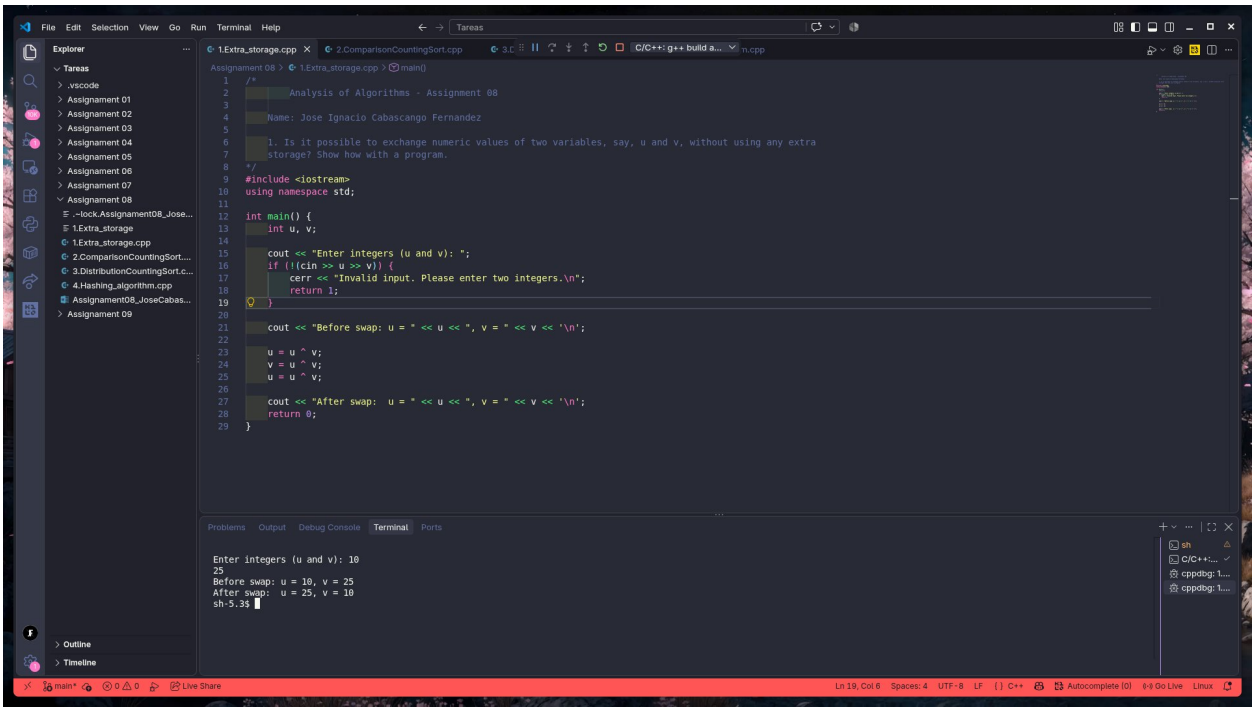
Assignament 08

Name: Jose Ignacio Cabascango Fernandez

Link GitHub: https://github.com/CodeWithJose1/Analysis-of-Algorithms/tree/main/Assignament-08/src

1. Is it possible to exchange numeric values of two variables, say, u and v, without using any extra storage? Show how with a program.

The program swaps two integer values using the bitwise XOR operator. XOR has the properties x ^ x = 0 and x ^ 0 = x, which allow the two values to be exchanged without a third variable. This approach avoids arithmetic overflow that can occur in the addition/subtraction swap technique.



Example

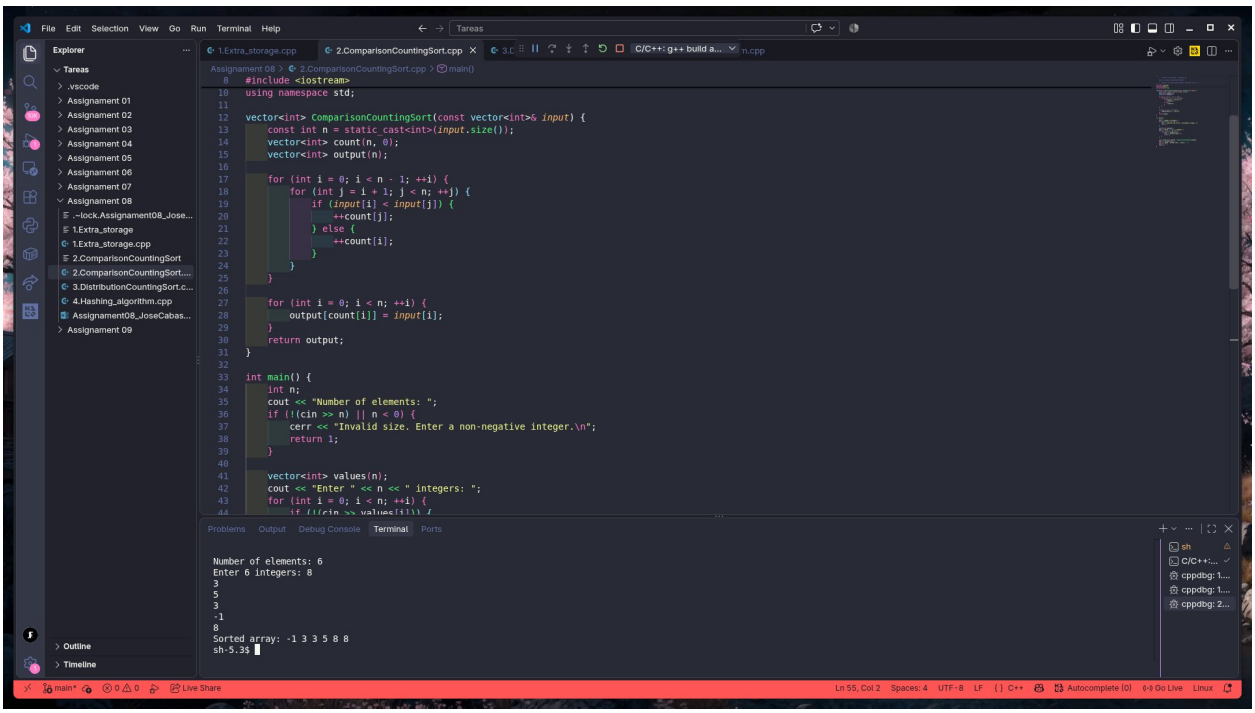
For u = 10 and v = 25, the output is u = 25 and v = 10.

Complexity

- Time: O(1), because exactly three XOR assignments are performed.
- Auxiliary space: O(1).

2. Implement the ComparisonCountingSort algorithm and test it.

For each pair of input values, the algorithm compares them and increments a rank counter. At the end, each value is placed at the position indicated by its rank. Equal values are assigned distinct positions by a consistent tie rule, preventing collisions in the output array.



Example

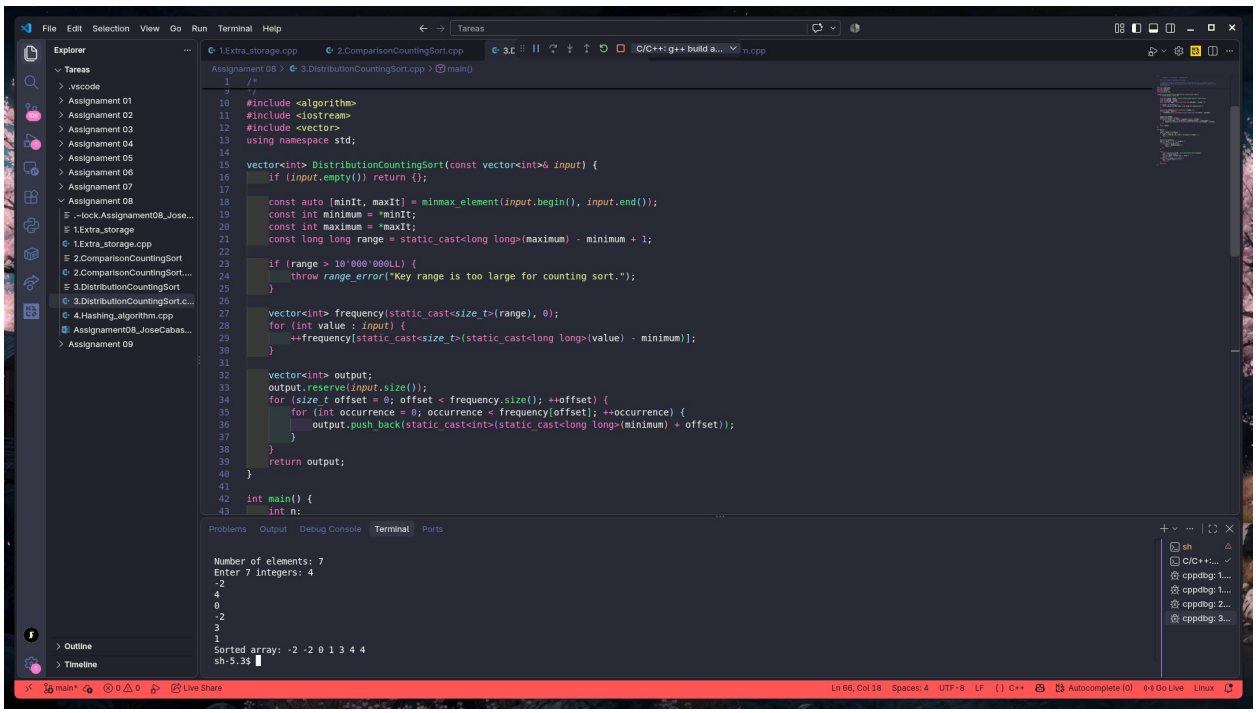
Input: [8, 3, 5, 3, -1, 8]
Output: [-1, 3, 3, 5, 8, 8]

Complexity

- Time: $\Theta(n^2)$, because every pair of elements is compared.
- Auxiliary space: $\Theta(n)$ for the rank and output arrays.

3. Implement the DistributionCountingSort algorithm and test it. Compare it with the ComparisonCountingSort from previous point. What are the differences of complexity and implementation?

The algorithm finds the minimum and maximum input values, allocates a frequency array for the integer range, counts each value, and emits each value according to its frequency. The offset value – minimum uallows negative keys to be handled.



Example

Input: [4, -2, 4, 0, -2, 3, 1]
Output: [-2, -2, 0, 1, 3, 4, 4]

Complexity

Let n be the number of input elements and k = maximum – minimum + 1 be the size of the key range.

- Time: Θ(n + k).
- Auxiliary space: Θ(n + k) in this implementation, including the output vector and frequency array.

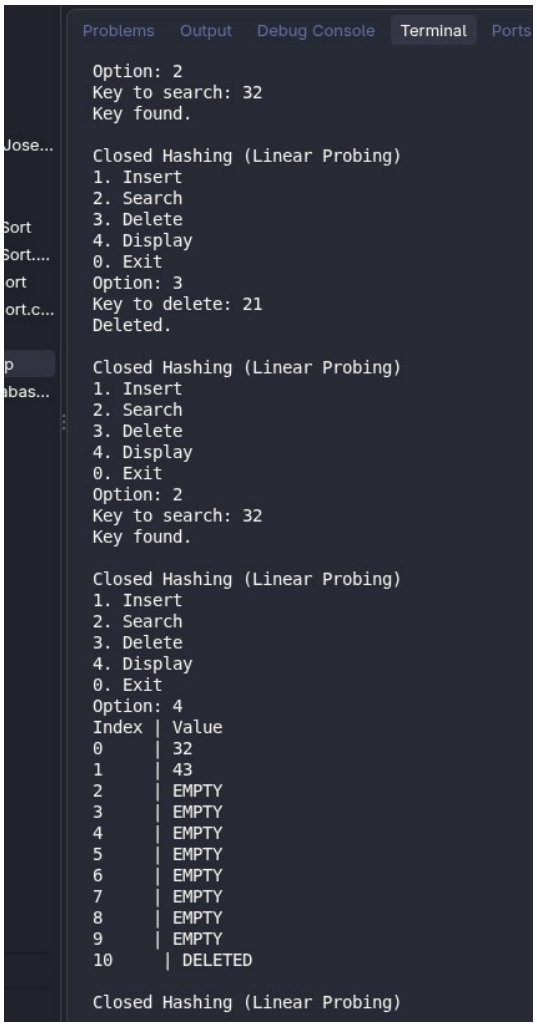
Comparison with Comparison Counting Sort

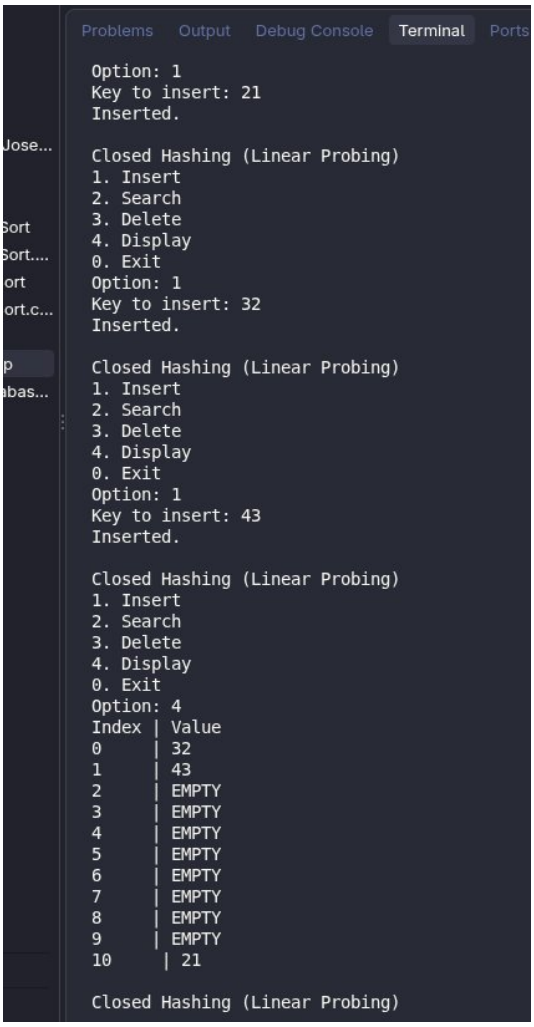
Feature	Comparison Counting Sort	Distribution Counting Sort
Main operation	Pairwise comparisons	Frequency counting
Time	Θ(n²)	Θ(n + k)
Extra space	Θ(n)	Θ(n + k)
Key assumptions	Comparable values	Integer keys with a manageable range
Best use	Small inputs or wide/unrestricted key values	Large inputs with a relatively narrow key range

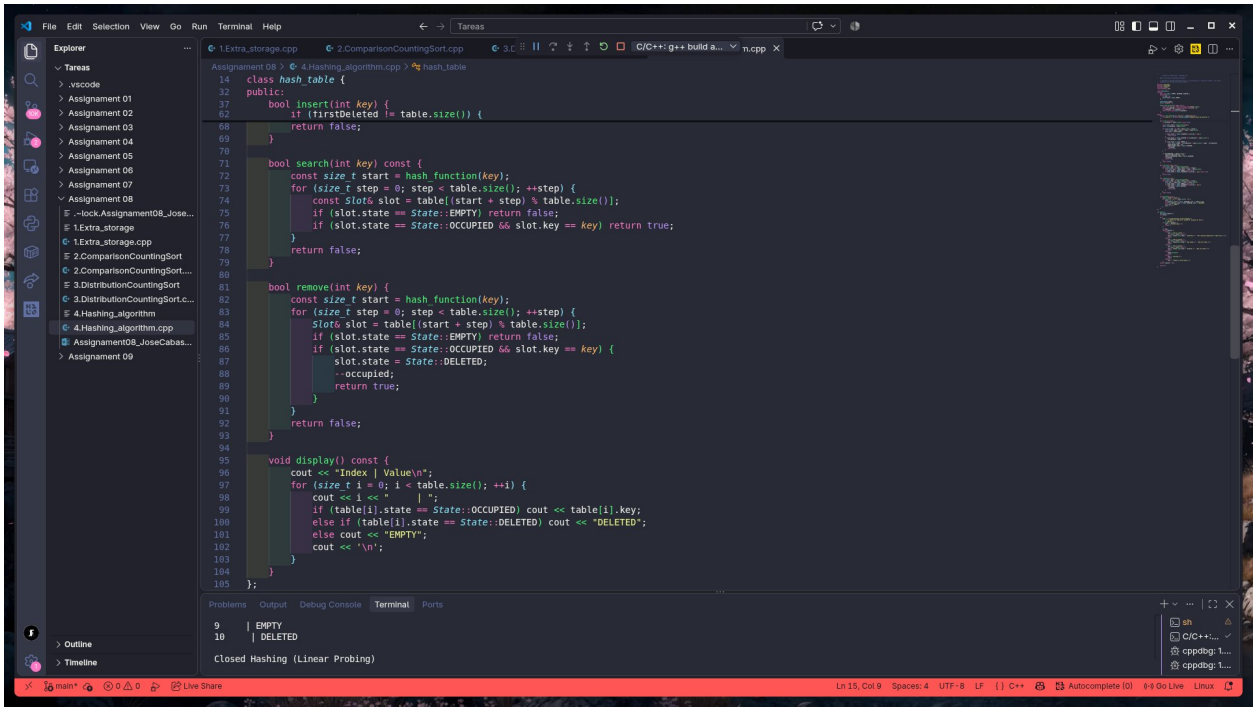
4. Implement a hashing algorithm using C++ (no dictionaries or libraries allowed). Use closed hashing as the collision resolution method.

The program implements a fixed-capacity hash table using closed hashing (open addressing) and linear probing. The hash function is h(key) = key mod m, where m = 11. If a slot is occupied, the algorithm checks subsequent slots cyclically until it finds the key or an available position.

The implementation supports insertion, search, deletion, and display. Deletion marks a slot as DELETED rather than EMPTY. This tombstone is important: making a deleted slot empty could cause a search to stop too early and miss a key stored farther along the probe sequence. Duplicate keys are rejected. Negative keys are normalized to a non-negative index.







Example collision

For capacity 11, keys 21, 32, and 43 all have hash index 10. Linear probing places them at indices 10, 0, and 1, respectively, if inserted in that order.

We calculate the initial position of each key:

Clave	Cálculo	Índice final
21	21 mod 11=10	10
32	32 mod 11=10	0
43	43 mod 11=10	1

Complexity

Let m be the table capacity.

- Expected time per insert/search/delete: O(1) under a well-distributed hash function and a moderate load factor.
- Worst-case time per operation: O(m), because the entire table may need to be probed.
- Space: Θ(m) for the table.